

R-Control IP4 – WebSocket Gateway (/ws)

This document describes the WebSocket interface of the R-Control IP4. It is typically used by UI/dashboard applications that want to switch relays and receive status/telemetry in real time.

1. Overview

- **Path:** /ws
- **Protocol:** WebSocket over HTTP or HTTPS (depending on the web server configuration)
- **Format:** JSON messages (text frames)
- **Max. concurrent clients:** 8
- **Initial state:** After a successful handshake, the device sends a one-time **snapshot**.

2. Connection Setup

2.1 URL

- ws://<device>:<port>/ws
- wss://<device>:<port>/ws

The port is the HTTP/HTTPS port (see REST API / device configuration).

2.2 Authentication

The WebSocket connection is **token-protected**. The token can be provided in two ways:

1. **HTTP header** (recommended for native clients)

Authorization: Bearer <token>

2. **Query parameter** (convenient for browser clients)

/ws?token=<token>

Error when token is missing/invalid

- HTTP status: 401 Unauthorized
- Header: WWW-Authenticate: Bearer realm="R-Control-IP4"
- Body: text Unauthorized

2.3 Token Expiry (Session)

- The gateway validates the token and manages an expiration time per connection.
- When the token expires, the device sends:

```
{"type": "session", "status": "expired"}
```

and then closes the connection.

Note: There is **no token refresh** in the gateway via response headers like in REST. Clients should log in again and reconnect when receiving `session/expired`.

3. Capacity Limit

If there are too many concurrent WebSocket clients, the handshake is rejected:

- HTTP status: `503 Service Unavailable`
- Content-Type: `application/json`
- Body:

```
{"error": "ws_capacity", "message": "Too many dashboard clients connected, please try again later."}
```

4. Message Format

- Each message is a JSON object with at least the field `type`.
- All message frames are text (`HTTPD_WS_TYPE_TEXT`).

5. Messages from the Device (Server → Client)

5.1 `snapshot` (Initial State)

Sent to a new client immediately after a successful handshake.

Example (structure):

```
{
  "type": "snapshot",
  "relays": [
    {"index": 0, "state": false, "impulse_active": false},
    {"index": 1, "state": true, "impulse_active": true, "impulse_type": "on",
"impulse_remaining_ms": 12000}
  ],
  "epoch": 1739836800,
  "sun_location_valid": true,
  "next_sunrise_epoch": 1739865600,
  "next_sunset_epoch": 1739901600,
  "ntc_raw": 1234,
  "ntc_status": 0,
  "ntc_mv": 1500,
  "ntc_temp_c": 22.75,
  "scheduler": {"runtime_rule_count": 3, "runtime_max_rules": 64, "...": "..."},
  "actions": {"runtime_rule_count": 1, "runtime_max_rules": 64, "...": "..."}
}
```

5.2 `time` (Periodic Time/Sensor Push)

The device sends a **time** event approximately **once per second**.

Example:

```
{
  "type": "time",
  "epoch": 1739836801,
  "sun_location_valid": true,
  "next_sunrise_epoch": 1739865600,
  "next_sunset_epoch": 1739901600,
  "ntc_raw": 1234,
  "ntc_status": 0,
  "ntc_mv": 1500,
  "ntc_temp_c": 22.75,
  "impulses": [
    {"index": 1, "type": "on", "remaining_ms": 11800}
  ]
}
```

5.3 relay (Relay State Changed)

Broadcast when a relay state changes:

```
{"type":"relay","index":1,"state":true}
```

5.4 impulse (Impulse Done)

When an impulse has expired:

```
{"type":"impulse","index":1,"status":"done"}
```

5.5 ntc (NTC Sensor Update)

On sensor updates:

```
{"type":"ntc","ntc_raw":1234,"ntc_status":0,"ntc_mv":1500,"ntc_temp_c":22.75}
```

5.6 pong (Response to Ping)

```
{"type":"pong","ts":123,"epoch":1739836801}
```

5.7 sched / sched_ack

- **sched**: scheduler configuration (incl. runtime metadata) + sun times
- **sched_ack**: status response after **sched_set**

Example **sched_ack**:

```
{"type":"sched_ack","status":"ok"}
```

5.8 **actions** / **actions_ack**

- **actions**: actions configuration (incl. runtime metadata)
- **actions_ack**: status response after **actions_set**

Example **actions_ack**:

```
{"type":"actions_ack","status":"ok"}
```

6. Messages to the Device (Client → Server)

6.1 **ping**

Client liveness / round-trip:

```
{"type":"ping","ts":123}
```

6.2 **relay_cmd**

Switch / blink relays:

Fields

- **index** (number): relay index (0...3)
- **cmd** (string):
 - **on**, **off**, **toggle**
 - **blink_start** (+ **period_ms**)
 - **blink_start_adv** (+ **on_ms**, **off_ms**)
 - **blink_stop**

Examples:

```
{"type":"relay_cmd","index":0,"cmd":"toggle"}
```

```
{"type":"relay_cmd","index":2,"cmd":"blink_start","period_ms":500}
```

```
{"type":"relay_cmd","index":2,"cmd":"blink_start_adv","on_ms":100,"off_ms":900}
```

6.3 impulse

Start an impulse (duration is clamped internally to **10...600000 ms**):

```
{"type":"impulse","index":1,"state":"on","duration_ms":30000}
```

state: on or off.

6.4 impulse_cancel

Cancel an active impulse:

```
{"type":"impulse_cancel","index":1}
```

6.5 time_sync

Set time (epoch seconds):

```
{"type":"time_sync","epoch":1739836800}
```

Notes:

- The system time is set.
- The RTC is only written if `epoch >= 2000-01-01`.
- After `time_sync`, the device immediately sends a `time` push.

6.6 sched_get / sched_set

Read/write scheduler.

- `sched_get` triggers a `sched` broadcast.
- `sched_set` expects:

```
{"type":"sched_set","data":{"... }}
```

Special case:

- If `data.astro` or `data.location` is present, it is persisted to `config.system.astro`.

6.7 actions_get / actions_set

- `actions_get` triggers an `actions` broadcast.
- `actions_set` expects:

```
{"type":"actions_set","data":{" ... }}
```

- The device then sends `actions_ack` and afterwards an updated `actions`.

7. Examples

7.1 Browser (JavaScript)

```
const ws = new WebSocket(`ws://${location.host}/ws?token=${token}`);
ws.onmessage = (ev) => console.log(JSON.parse(ev.data));
ws.onopen = () => {
  ws.send(JSON.stringify({ type: "ping", ts: Date.now() }));
  ws.send(JSON.stringify({ type: "relay_cmd", index: 0, cmd: "toggle" }));
};
```

7.2 Python (websockets)

```
import asyncio, json
import websockets

async def main():
    uri = "ws://r-control-ip4.local/ws?token=" + TOKEN
    async with websockets.connect(uri) as ws:
        await ws.send(json.dumps({"type":"ping","ts":1}))
        async for msg in ws:
            print(json.loads(msg))

asyncio.run(main())
```

8. Security Notes

- The WebSocket handshake is token-protected.
- After the handshake, **no additional permission checks** are performed per message. Any successfully authenticated user can therefore (as of today) also trigger write actions (relays, scheduler, actions, time).
- Recommendation: Only issue tokens to clients that are explicitly allowed to use these control functions; otherwise extend roles/permissions server-side.

9. Troubleshooting

- `401 Unauthorized` during handshake: token missing/invalid.

- `503 ws_capacity`: too many concurrent WS clients; make sure to properly close old sessions in the client.
- `{"type": "session", "status": "expired"}`: token expired → log in again and reconnect.